# CO215 Assignment-5

**Roll number:** CSB23017

**Name:** Raktim Ranjan Handique

**Session: Spring** **Year:** 2025

**Objective:**

1. To get acquainted with reading integer input from user
2. To understand and use indirect addressing mode
3. To become familiar with loops and transfer of control instructions

**Exercises:**

1. Create a file of the µP 8086 ALP program below to read an array of single-digit integers from the user and compute the sum.

```
CR          EQU     0DH
LF          EQU     0AH
MAX_SIZE    EQU     5

.MODEL      SMALL
.STACK      100H
.DATA

array       DW  MAX_SIZE DUP(?)
sum         DW  0
prompt_msg  DB  'Enter 5 non-zero single digit positive
integers.', CR, LF, CR, LF, '$'
output_msg  DB  CR, LF, 'Sum of the integers is: $'
ten         DB  10
.CODE

main    PROC
        .STARTUP

        lea     dx, prompt_msg
        mov     ah, 09h
        int     21h

        mov     bx, OFFSET array
        mov     cx, MAX_SIZE
        sub     dx, dx

rd_loop:
        sub     ax, ax
        mov     ah, 01h
        int     21h
        sub     al, 30h
```

```
        mov     ah, 0

        mov     [bx], ax
        add      bx, 2
        loop     rd_loop

compute_sum:
        mov     bx, OFFSET array
        mov     cx, MAX_SIZE
        mov     ax, 0

sum_loop:
        add     ax, [bx]
        add     bx, 2
        loop    sum_loop

        mov     sum, ax

        lea     dx, output_msg
        mov     ah, 09h
        int     21h

        sub     dx, dx
        mov     ax, sum
        div     ten
        mov     bx, ax
        add     bh, 30h
        add     bl, 30h

        mov     dl, bl
        mov     ah, 02h
        int     21h
        mov     dl, bh
        mov     ah, 02h
        int     21h

.EXIT
        mov     ah, 4ch
        mov     al, 0
        int     21h

main    ENDP
            END main
```

2. **List the different addressing modes used in the program**
   **:** Addressing Modes Used in the 5-Digit Sum Program

1. **Immediate Addressing Mode**

   mov ah, 09h

```
mov al, 0
mov bx, 10
```

The operand is specified directly in the instruction. Example: `09h`, `0`, and `10` are used as immediate values.

2. **Direct Addressing Mode**

```
mov bx, OFFSET array

mov sum, ax
```

The instruction accesses memory locations directly using a label (`array`, `sum`).

3. **Register Addressing Mode**

```
mov ax, 0

mov bx, ax
```

The operand is in a register, and data is transferred between registers (e.g., `ax`, `bx`, `cx`).

4. **Indirect Addressing Mode**

```
mov [bx], dx

add ax, [bx]
```

The memory address is stored in `bx`, and the data is accessed indirectly through `[bx]`.

5. **Indexed Addressing Mode**

Not explicitly used in the program, but if an index register like `si` or `di` was used for array traversal, it would fall under this category.

3. **Rewrite the program to allow reading any number of integers terminated by 0 (up to a maximum of 20)**

```
:
CR          EQU       0DH
LF          EQU       0AH
MAX_SIZE      EQU       20

.MODEL        SMALL
.STACK        100H
.DATA

array         DW   MAX_SIZE DUP(?)
sum           DW   0
prompt_msg    DB   'Enter 20 non-zero single digit positive
integers.', CR, LF, CR, LF, '$'
output_msg    DB   CR, LF, 'Sum of the integers is: $'
ten           DB   10
.CODE

main      PROC
          .STARTUP

          lea       dx, prompt_msg
          mov       ah, 09h
          int       21h

          mov       bx, OFFSET array
          mov       cx, MAX_SIZE
          sub       dx, dx

rd_loop:
          sub       ax, ax
          mov       ah, 01h
          int       21h
          sub       al, 30h

          cmp       al, 0
          je        compute_sum

          mov       ah, 0

          mov       [bx], ax
          add        bx, 2
          loop       rd_loop


compute_sum:
          mov       bx, OFFSET array
          mov       cx, MAX_SIZE
          mov       ax, 0

sum_loop:
          add       ax, [bx]
```

```
        add     bx, 2
        loop    sum_loop

        mov     sum, ax

        lea     dx, output_msg
        mov     ah, 09h
        int     21h

        sub     dx, dx
        mov     ax, sum
        div     ten
        mov     bx, ax
        add     bh, 30h
        add     bl, 30h

        mov     dl, bl
        mov     ah, 02h
        int     21h
        mov     dl, bh
        mov     ah, 02h
        int     21h

.EXIT
        mov     ah, 4ch
        mov     al, 0
        int     21h

main    ENDP
            END main
```

4. **Rewrite the program to allow reading two digit integers**

```
CR      EQU     0DH

LF      EQU     0AH

MAX_SIZE    EQU     20


.MODEL      SMALL

.STACK      100H

.DATA


array       DW  MAX_SIZE DUP(?)
```

```
sum          DW   0

prompt_msg   DB   'Enter 20 non-zero two-digit positive integers.',
CR, LF, CR, LF, '$'

output_msg   DB   CR, LF, 'Sum of the integers is: $'

ten          DW   10

hundred      DW   100


.CODE


main     PROC

         .STARTUP


         lea      dx, prompt_msg

         mov      ah, 09h

         int      21h


         mov      bx, OFFSET array

         mov      cx, MAX_SIZE

         sub      dx, dx


rd_loop:

         sub      ax, ax


         mov      ah, 01h

         int      21h

         sub      al, 30h

         mov      ah, 0

         mul      ten

         mov      dx, ax


         mov      ah, 01h
```

```
        int     21h
        sub     al, 30h
        mov     ah, 0
        add     dx, ax

        mov     [bx], dx
        add     bx, 2
        loop    rd_loop

compute_sum:
        mov     bx, OFFSET array
        mov     cx, MAX_SIZE
        mov     ax, 0

sum_loop:
        add     ax, [bx]
        add     bx, 2
        loop    sum_loop

        mov     sum, ax

        lea     dx, output_msg
        mov     ah, 09h
        int     21h

        mov     ax, sum
        mov     cx, 0
        mov     bx, 10

convert_loop:
```

```
        mov     dx, 0
        div     bx
        push    dx
        inc     cx
        test    ax, ax
        jnz     convert_loop

print_digits:
        pop     dx
        add     dl, 30h
        mov     ah, 02h
        int     21h
        loop    print_digits

.EXIT
        mov     ah, 4Ch
        mov     al, 0
        int     21h

main    ENDP
        END main
```